

SRTM Board Monitoring GUI

WinCC OA Setup and Configuration Guide

Prepared by: Nathan Herling
M.S. Candidate, Data Science
University of Arizona
nth@arizona.edu

Assisted by: Claude (Anthropic)

February 2026

Table of Contents

Section	Page
1. OPC UA Node Discovery Tools	2
2. Creating a New WinCC OA Project	3
3. OPC UA Driver Configuration	5
4. Datapoint Configuration	6
5. GUI Panel Creation	7
6. Key File Locations	11
7. Quick Reference Commands	11

Overview

This guide documents the process of creating a monitoring GUI for the SRTM (Slow Rate Timing Module) board using WinCC OA. The SRTM board exposes sensor data via an OPC UA server, which WinCC OA connects to for real-time visualization.

Component	Details
SRTM Board	IP: 192.168.0.117
OPC UA Server	opc.tcp://192.168.0.117:4841
WinCC OA Server	eepp-bigm3.physics.arizona.edu
WinCC OA Version	3.19
Project Name	SRTM_Monitor_v3

1 OPC UA Node Discovery Tools

Before configuring WinCC OA, it's useful to browse the available OPC UA nodes on the SRTM board. Two Python scripts are available for this purpose:

Script	Output	Description
browse_opcua.py	opcua_nodes.csv	CSV format for analysis
browse_opcua_to_file_text.py	opcua_nodes.txt	Formatted text tree view

Location: /home/naherlin/ on eepp-bigm3

Usage:

```
/usr/bin/python3 browse_opcua.py
```

1.1 Exploratory Script: Reading a Single Node

Before configuring WinCC OA, you can verify OPC UA connectivity using this simple script that reads the FPGA temperature directly:

File: read_fpga_temp.py

Location: /home/naherlin/

```
#!/usr/bin/python3
from opcua import Client
import time

OPCUA_URL = "opc.tcp://192.168.0.117:4841"
NODE_ID = "ns=2;s=SRTM.FPGA_temp"
PING_TIME = 10 # seconds

client = Client(OPCUA_URL)
try:
    client.connect()
    node = client.get_node(NODE_ID)
    for i in range(PING_TIME):
        value = node.get_value()
        ts = time.strftime('%Y-%m-%d_%H:%M:%S')
        print(f"{ts}|_FPGA_temp={value}")
        time.sleep(1)
except Exception as e:
    print("ERROR:", e)
finally:
    client.disconnect()
```

Usage:

```
/usr/bin/python3 read_fpga_temp.py
```

Tip: This script confirms that the OPC UA server is accessible and the node path (ns=2;s=SRTM.FPGA_temp) is correct before configuring WinCC OA.

2 Creating a New WinCC OA Project

2.1 Starting the Project Administrator

Launch the Project Administrator from terminal:

```
startPA
```

This starts the Project Administrator window.

2.2 Method A: Quick Start

This is the preferred method because it will short circuit the need to create datapoints and install JCOP.

Step 1: Create a new project from the PA console.

1. Click the **New Project** button (blank document icon in toolbar)
2. Select **Distributed project**
3. Enter a Project Name and use the default path: `/home/monitor/WinCC/SRTM_SandBox/`
4. Change the System name to: `ATLLARLASPSRTM`
5. Click **OK**

Important: Use “Distributed project” type, NOT “Standard project”. Standard projects cause errors in later configuration steps. Datapoints will have a `ATLLARLASPSRTM:` prefix.

This will have created a skeleton project in the `/home/monitor/WinCC/SRTM_SandBox` directory.

Step 2: Copy a working project into the new project directory. For now the `SRTM_Monitor_v4` works well.

```
cd ~monitor/WinCC/SRTM_SandBox
cp -r SRTM\_Monitor_v4 <Project Name>
cd NewProject
```

Step 3: Update config/config file

```
sed -i 's/SRTM_Monitor_v4/<Project Name>/g' config/config
```

Step 4: Optional: Change pmonPort to avoid conflict

```
sed -i 's/pmonPort = 4999/pmonPort = 5001/g' config/config
```

Step 5: Clear old log files

```
rm -rf log/*
```

Step 6: Verify config/config

```
cat config/config
```

Setting	Value
proj_path	/home/monitor/WinCC/SRTM_SandBox/<Project Name>
pmonPort	5001 (unique port)
distributed	1

Step 7: Stop the original project before starting

The copied project shares default port configurations with the original. You must stop the original project first:

```
pkill -u $USER -f <Project Name>'
# Verify processes stopped:
ps aux | grep -i wincc | grep -v grep
```

Step 8: Refresh the Project Administrator panel (started by startPA) — the new project should appear in the list. Start it. If you are asked for a password, just click "OK".

Note: If you need to run both projects simultaneously, add unique ports to config/config:
dataPort = 4898
eventPort = 4899
Otherwise, the default ports will conflict.

FYI: To check settings from an existing project:
cat /WinCC.Projects/<ProjectName>/config/config

Old log files may contain references to the original project name — this is harmless. Use `grep -r 'OldName' . 2>/dev/null` to verify no critical files are affected.

2.3 Method B: Building the project from scratch (not the preferred method)

This method allows you to start building the project from scratch. It assumes that you have already created a skeleton project and need to connect the project to the SRTM and create datapoints.

Step 1: Create a new project from the PA console.

```
Click the New Project button (blank document icon in toolbar)
Select Distributed project
Enter a Project Name and use the default path
    /home/monitor/WinCC/SRTM_SandBox/
Change the System name to: ATLLARLASPSRTM
Click OK
```

Important: Use "Distributed project" type, NOT "Standard project". Standard projects cause errors in later configuration steps. Datapoints will have a ATLLARLASPSRTM: prefix.

You can go to the Panel Administrator window and start your project by clicking on the name of the new project and then clicking on the "Start Project" button (it looks like a green stoplight). This will start the project console. If you get a pop up window asking for a password, just click "OK"

Step 2: Install JCOP.

See: jcop.web.cern.ch

Get the files

```
cd /home/monitor/WinCC/SRTM_SandBox/<Project Name>
wget https://edms.cern.ch/file/1055675/9.3.0/jcop-framework-9.3.0.1.
    zip
wget https://edms.cern.ch/file/1085149/9.3.1/fwInstallation-9.3.1.zip
```

Unzip the files

```
unzip fwInstallation-9.3.1.zip
unzip jcop-framework-9.3.0.1.zip
```

Highlight the GEDI window (it should have opened when you started the project)

If the window did not open, click on the Project Console.
You should see a line called User Interface.
Click on this line, and then click on the start button to the right (and top) of the manager section of the window

In the GEDI window, find the line that says Panel

Expand the Panels line and search for fwiInstallation
Expand the fwInstallation section and search for a panel called fwInstallaion.pnl
Double click on this line and click the "Save and Run in QuickTest Module" (it looks like a diskette with a green arrow)

This will open a new window that will allow you to install the JCOP modules

Choose the folder button and navigate to the location of the jcop-framework directory, click on the Scan recursively box
Click in the Install area of the line that says jcop-framework install in a subdirectory of your project (i.e. fwComps)
Also install the fwCore option
Cancel installation of other hardware stuff (i.e. Wiener)
Click on the "Install ..." button

Step 3: Connect to the SRTM OpcUa server We have been configuring the OPC UA driver as driver number 10 (to avoid conflict with the simulation driver on number 1).

Edit config/progs file:

```
WCCOAopcua | always | 30 | 1 | 1 | -num 10
```

Configuring the OPC UA Connection (GUI)

Open SysMgm --> Driver OPC --? OPC UA Client
Click "Create" to add a new connection (or select existing)
Configure the following settings:

Setting	Value
Connection Name	SRTM
Driver Number	10
Server URL	opc.tcp://192.168.0.117:4841
Security Policy	None
Authentication	Anonymous

Check the "Active" checkbox
Click "Apply"
Verify "Status OPC UA Server 1" ‘‘Connected’’ and ‘‘Running’’
Click "Browse" to verify you can see the SRTM nodes
Click "OK"

OPC UA Client Configuration Window:

WinCC_OA(1): OPCUA client configuration (dist_1 - SRTM_Monitor_v3; #1)...

Connection

SRTM OPCUA_D2 [Remove] [Create]

Device Description: dist_1:_SRTM_OPCUA_D2.

Settings

Reconnect Timer: 10 [s] Driver number: 2

Connection [Advanced]

Server

☒ Active URL/URI: opc.tcp://192.168.0.117:4841

Authentication

Anonymous

Security

None None

Redundant Server

☐ Active URL/URI: opc.tcp://

Commands [Browse] [General Query]

Subscriptions [Manage]

Status OPC UA Server 1

eepp-bigmem3.physics.arizona.edu

State Connected Server Running

Status OPC UA Server 2

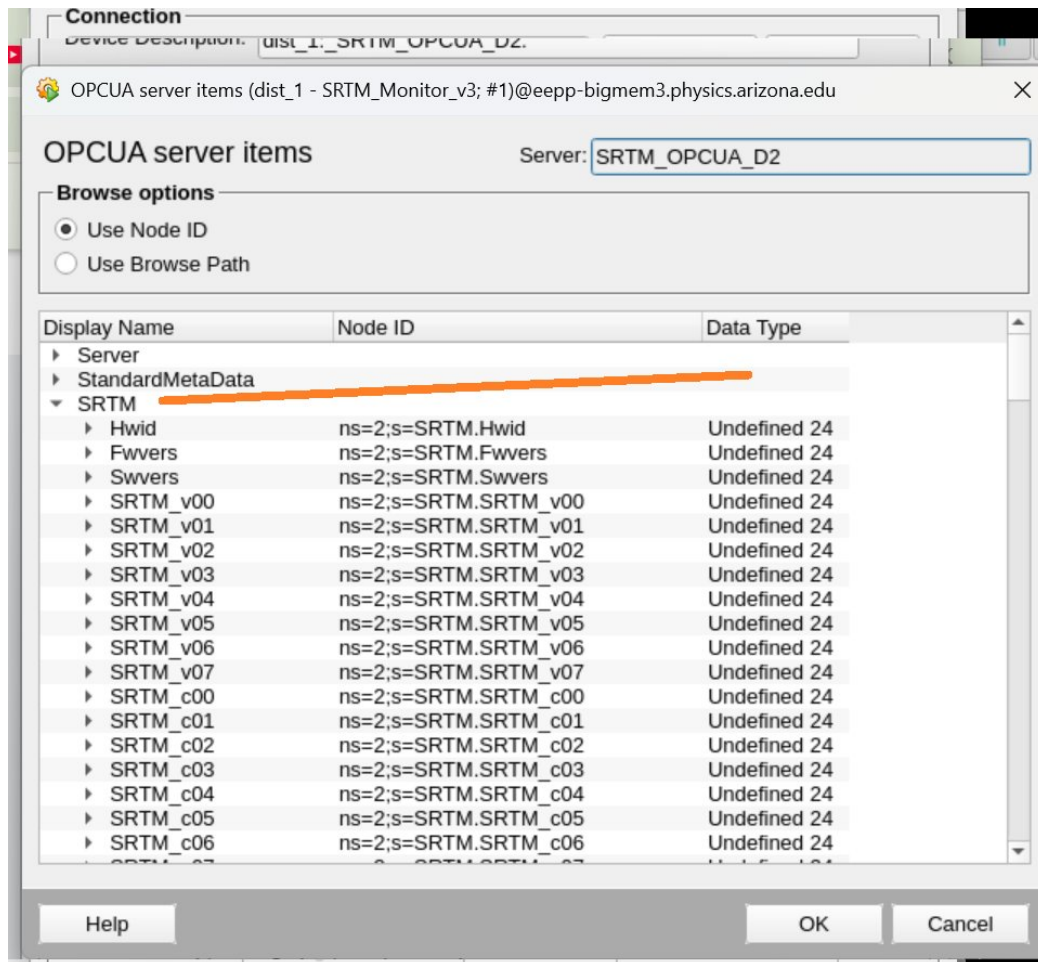
eepp-bigmem3.physics.arizona.edu

State Not connected Server

[Help] [OK] [Apply] [Cancel]

Creating a Datapoint

Browse OPC UA Server Items:



Step 4: Creating Datapoints

In WinCC datapoints are the data that are passed from the OpcUa server to WinCC. A datapoint element is a group of similar datapoints. For the SRTM the datapoint element is called SRTM. The datapoint element SRTM is part of a datapoint type called SRTMRegs. There are two ways to create datapoints: manually and using Cacophony. The latter method is preferred since it automatically generates all of the datapoints from the quasar project. Since there are over 100 datapoints, this is quite painful to do manually.

Creating Datapoints with Cacophony (preferred method)

Some useful information can be found here:

1. See <https://www.youtube.com/watch?v=g2LWOx1BufI> at 13:40 or so
2. See <https://github.com/quasar-team/Cacophony>

Download the cacophony package and run it to create two scripts for defining the SRTM datapoints.

```
cd quasar-srtm-built (this is the top directory of the quasar project
used to create the quasar OpcUa server)
git clone https://github.com/quasar-team/Cacophony.git
pip install colorama (this might be necessary if it is not already
installed)
python Cacophony/generateStuff.py --dpt_prefix SRTM --server_name SRTM
--driver_number 10 --subscription SRTM_SUBSCRIPTIONS
cd Cacophony/generated
cp *.ctl ~/WinCC/SRTM_SandBox/<Project Name>/scripts/libs/
```

You should be able to see the files appear in Gedi under <Project Name> → Libraries. We will now create a panel that will allow us to run these scripts.

Create a new panel using the first button in Gedi (looks like a page with a + sign)
 Add a pushbutton (about the middle of the first row of buttons)
 To change the name of the button, change the "Button Label" in the Property Editor window that should open when you click on the button in the panel.
 In the Script panel (Part of the Property Editor) click on the script button next to the Clicked event (looks like "{..}")
 Add the line: #uses "createDpts.ctl" at the top of the script
 Add the line: createDpts();
 as part of the main(..) body (i.e. between the curly braces)
 Save and run this panel (choose a name to save the file such as CreatedDPTs.pnl)
 In the QuickTest window, click on the button. You should see an entry in the log file that says "creating DPTs for class Regs"
 If you open the Para window (six button on the top row), you should now see a DPT type called SRTMRegs.

Add another pushbutton to the same panel (I used the Button Label "Parse Config")

In the Script panel click on the script button next to Clicked event
 At the top add: #uses "configParser.ctl"
 In the body of the script add:
 parseConfig("/home/monitor/quasar-srtm-built/build/bin/config.xml",
 true,
 true,
 true);
 Save the script and run the panel with QuickTest

You should see a new window pop up with the added button displayed.

Click on the Parse Config button (you should see some messages in the log panel)
 This should produce a set of DPs that list all of the variables available from the SRTM (Open a Para window and scroll down until you see the datapoint type SRTMRegs)

You can skip to the next section

Creating Datapoints Manually (not preferred)

The first step is to create a datapoint type.

Open a Para window (if you don't already have one open)
 Click on the Para button (first row of buttons, currently sixth from the left)
 Create a new Datapoint Type (i.e. SRTM_float_data)
 Right click on ATLLARLASPSRTMat the top of the datapoint type tree
 Choose Create datapoint type
 Right click on the folder next to newDpType
 Choose the Element-type (i.e. float)
 Double click on newDpType to highlight it and change the name (i.e. SRMT_float_data)

Now you will create each datapoint as part of the newly created datapoint type.

Right-click on SRTM_float_data --> Create new datapoint
 Choose a name corresponding to the SRTM data (i.e. F11_tempC)
 Right click on your newly created datapoint
 Choose Insert config --> Periphery address
 Click on the _address config, choose the OPCUA Client Driver type and click Configure
 Choose the Server SRTM

Select the SRTM_SUBSCRIPTIONS
Click on "Get Item" and choose from the variables from the in the drop down menu
Select the "Input" Direction
Click on the "Address active" button and hit "OK"

Property	Value
Name	SRTM_FPGA_temp
Type	ExampleDP_Float

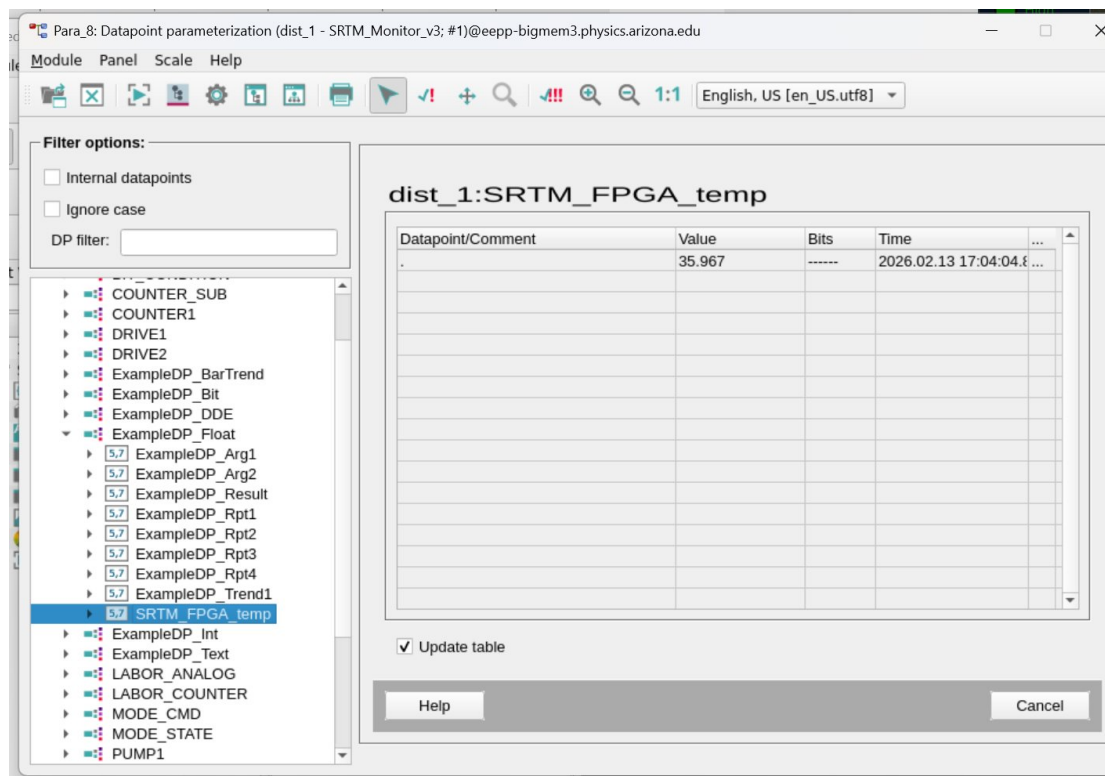
Address Configuration (Peripheral Address) (not preferred)

The address maps the datapoint to the OPC UA node on the SRTM board:

Property	Value
Driver Number	2
Address Type	OPCUA
Reference	SRTM.OPCUA_D2\$ns=2;s=SRTM.FPGA_temp
Direction	Input (1)

Tip: The OPC UA node address format is: ns=2;s=SRTM.<NodeName>

PARAM Window showing SRTM_FPGA_temp datapoint with live value:



3 GUI Panel Creation

3.1 Opening the Graphics Editor (GEDI)

GEDI is typically already open when you start the project.

Or from terminal:

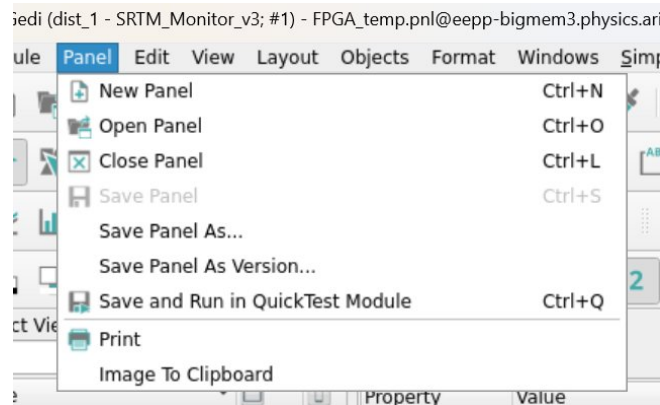
```
/opt/WinCC_OA/3.19/bin/WCCOAgedi -PROJ SRTM_Monitor_v3 &
```

3.2 Creating a New Panel

1. **Panel** → **New Panel** (Ctrl+N)
2. Set panel size (e.g., 500 x 400) in the Property Editor
3. **Panel** → **Save Panel As...**
4. Save as: FPGA_Monitor.pnl

Location: /WinCC_Projects/SRTM_Monitor_v3/panels/

Panel Menu:



3.3 Adding a Value Display

1. Select Text tool (T) from toolbar
2. Draw text box on panel
3. Select the text element
4. In Property Editor → Event → Initialize → click script button
5. Check “Display value” and click OK
6. Select datapoint: SRTM_FPGA_temp

3.4 Adding an LCD Number Display

1. **Objects** → **More Objects** → **LCD Number**
2. Draw the LCD area on the panel
3. Select the LCD widget, then in Property Editor find **Event** → **Initialize**
4. Click the script button to open the Script Editor
5. Enter the following Initialize script:

LCD Initialize Script:

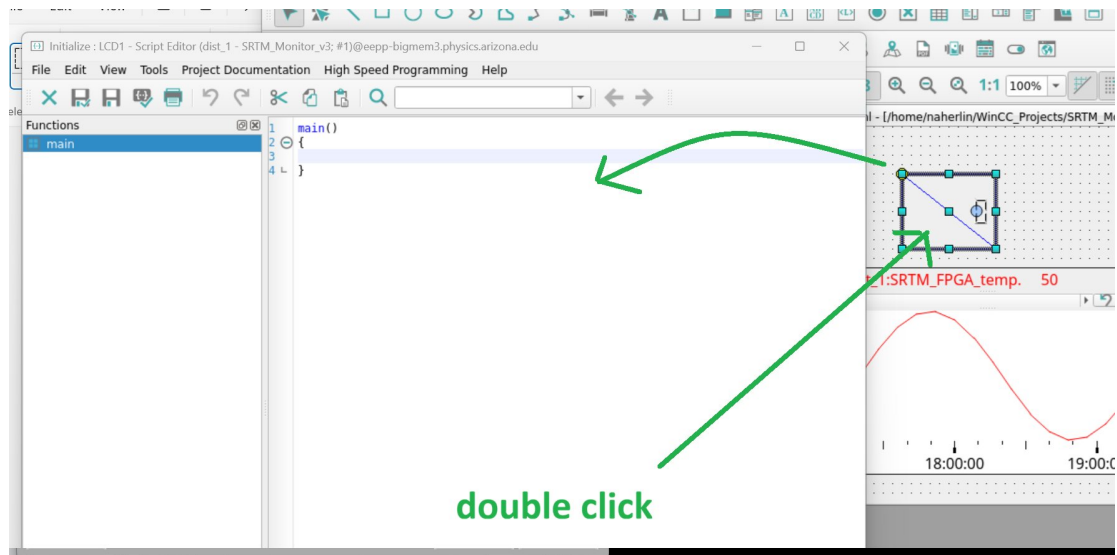
```
main()
{
    dpConnect("updateLCD", "dist_1:SRTM_FPGA_temp::_online::_value");
}

void updateLCD(string dp, float value)
{
    this.value = value;
}
```

6. Save the script (File → Save) and close the Script Editor

Note: Each widget requires a datapoint binding or script to display live data. Double-clicking a widget opens its configuration. The Script Editor (shown when clicking script icons) allows custom control logic using the `dpConnect` function.

Widget Script Editor and Live Panel:

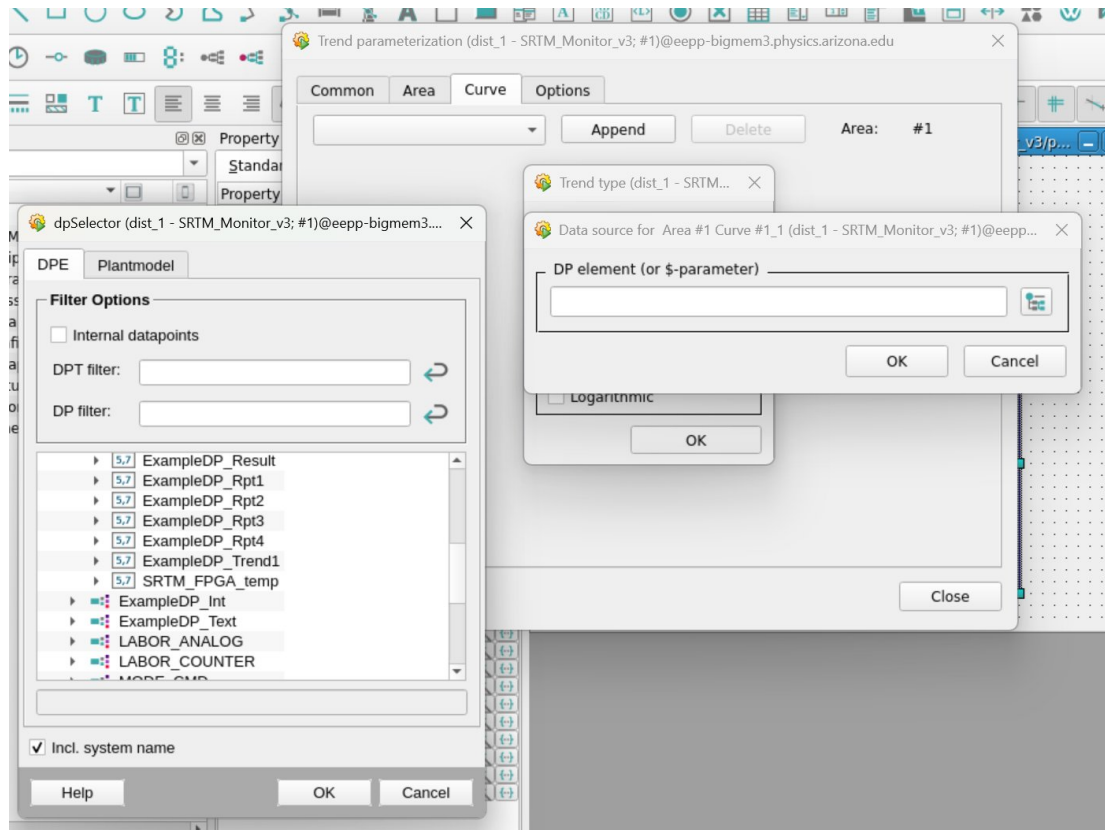


3.5 Adding a Trend (Time Series Chart)

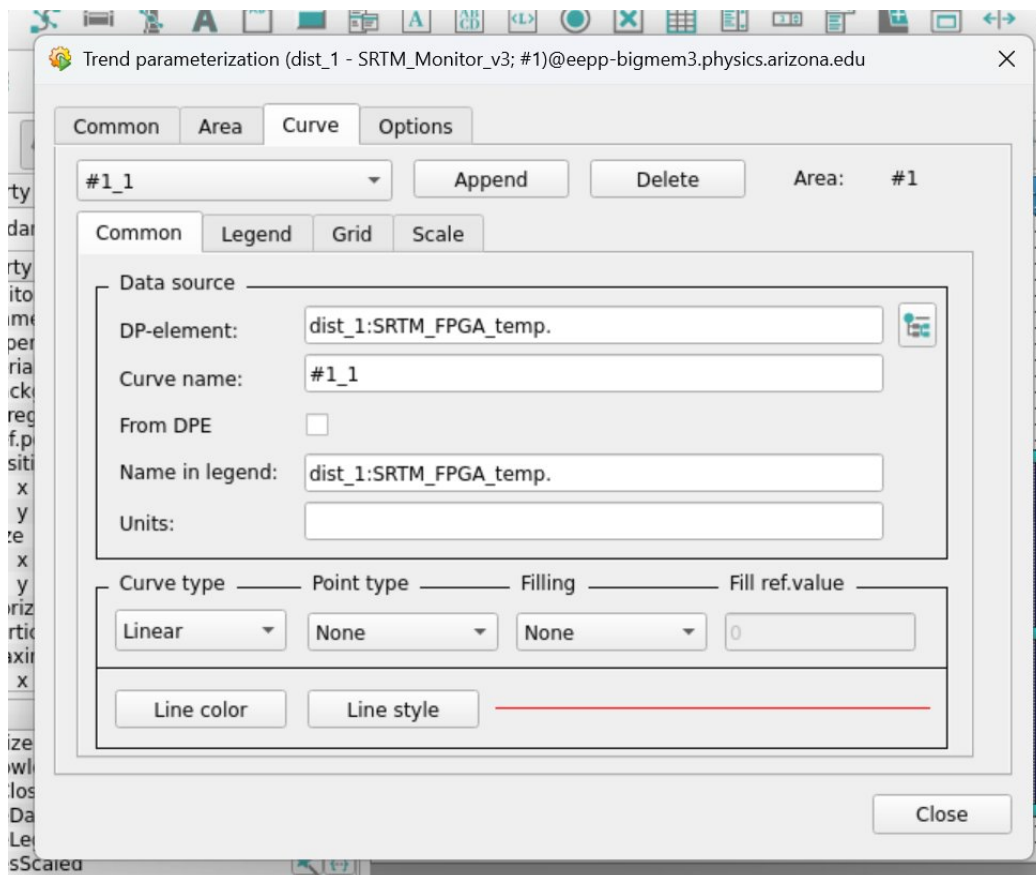
1. Objects → More Objects → Trend
2. Draw trend area on panel
3. Double-click to configure:

Tab	Setting	Value
Common	Display time range	0 Days, 0 Hours, 5 Minutes
Curve	Datapoint	SRTM.FPGA_temp.
Scale (Y)	Auto scale	Checked (or set Min/Max)

Selecting the Datapoint for the Trend:



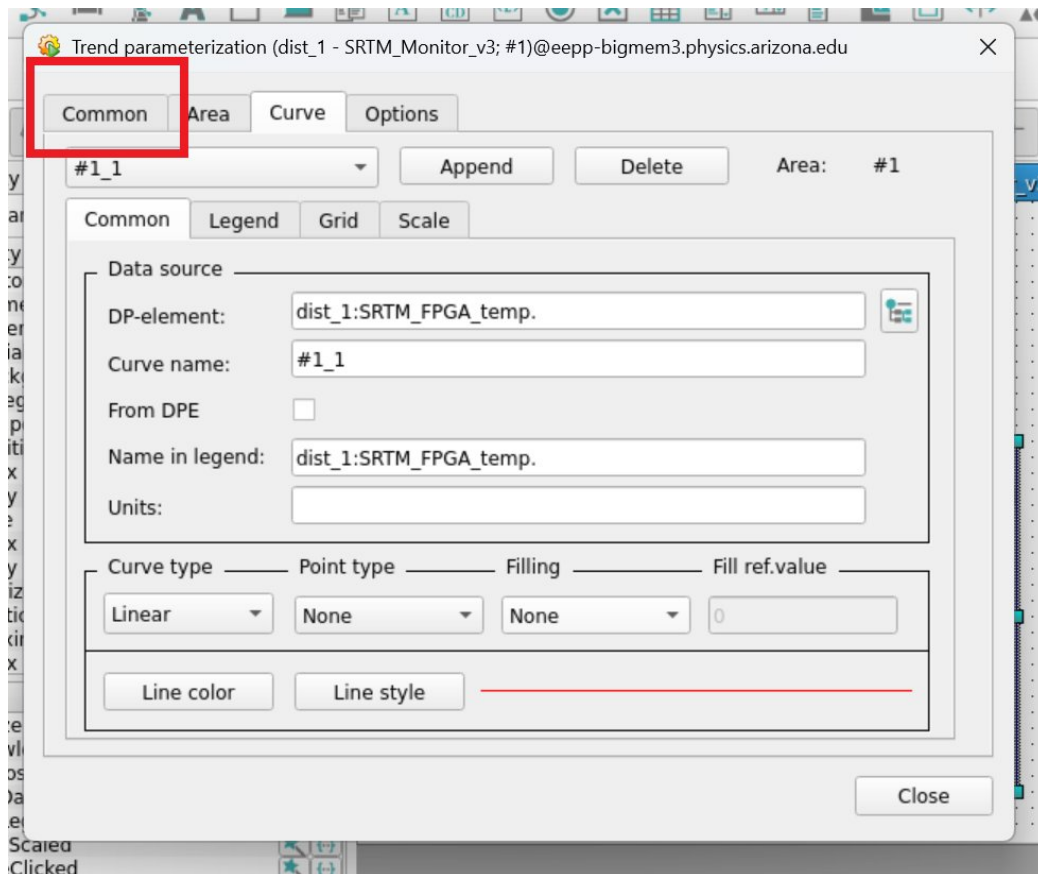
Curve Configuration with Datapoint Bound:



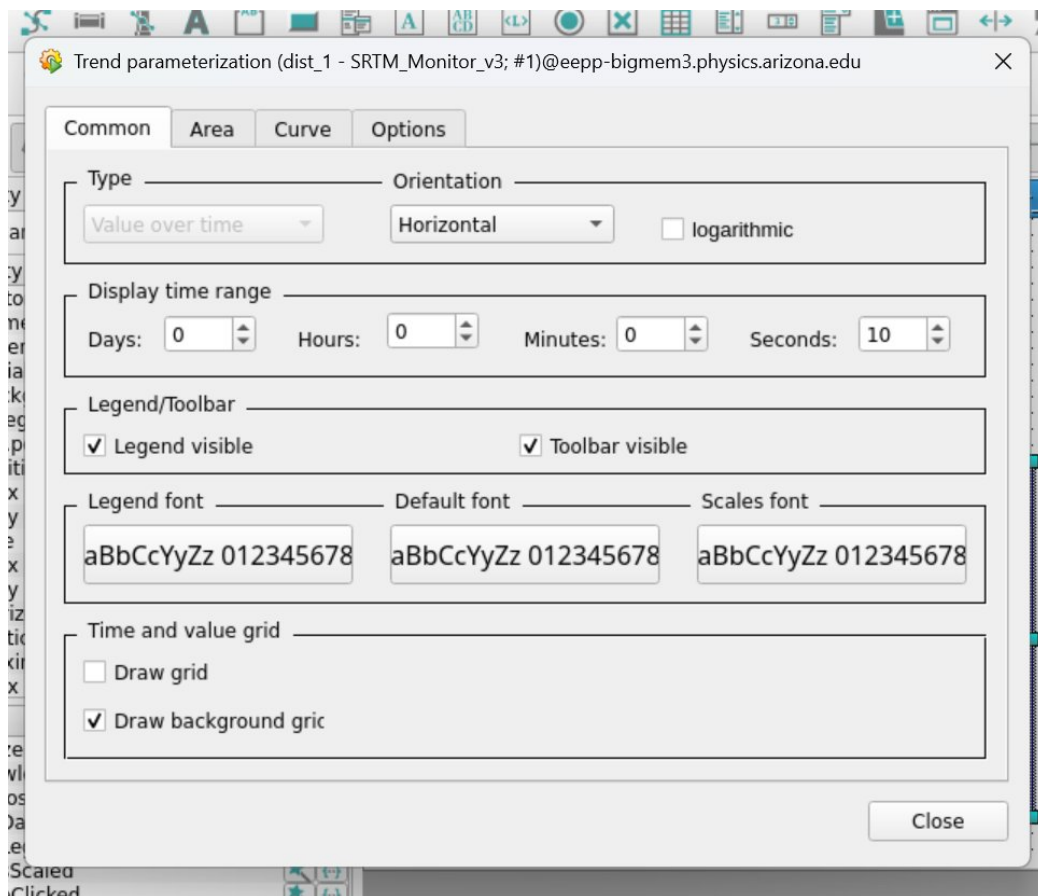
3.6 Configuring the Time Scale (X-Axis)

The Trend parameterization has two “Common” tabs — one at the top level and one under the Curve tab. Use the **top-level Common** tab to set the display time range.

Click the top-level Common tab (highlighted):



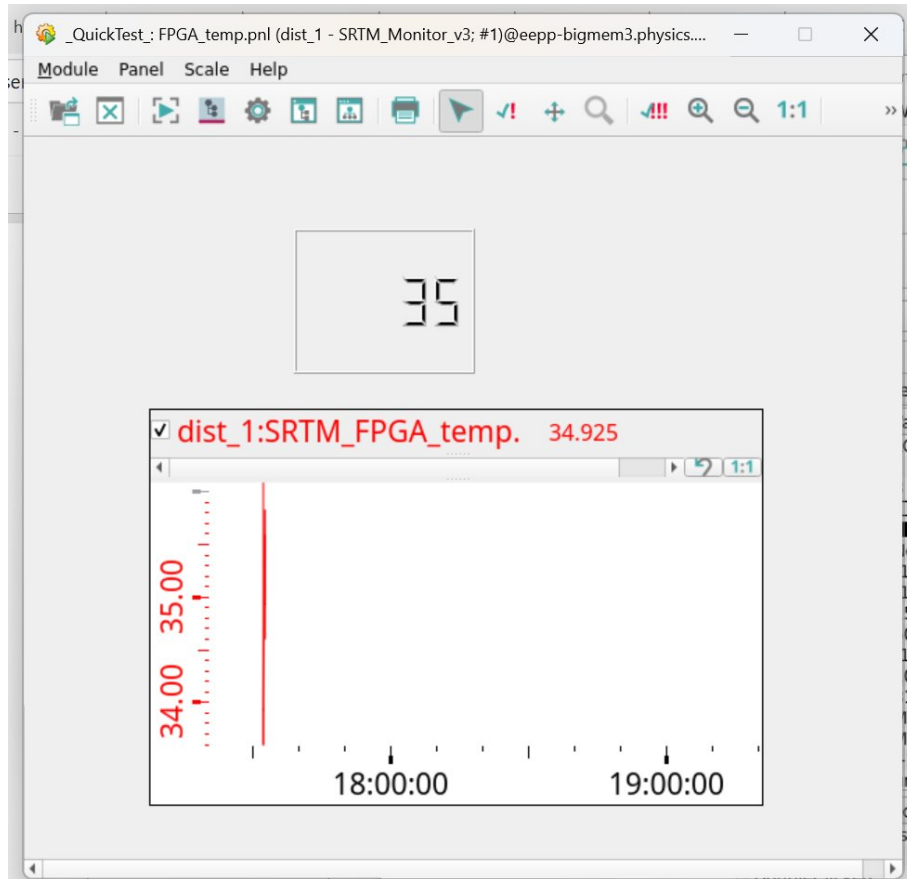
Set Display time range (e.g., 10 seconds):



3.7 Testing the Panel

1. Save panel: Ctrl+S
2. Quick Test: F5 or Panel → Save and Run in QuickTest Module (Ctrl+Q)

Working GUI (before time-scale adjustment):



After Y-bounds fixed and time-scale set to 10 seconds:

4 Key File Locations

File/Directory	Path
WinCC OA Installation	/opt/WinCC_OA/3.19/
Project Directory	/WinCC_Projects/SRTM_Monitor_v3/
Config Files	/WinCC_Projects/SRTM_Monitor_v3/config/
Panels	/WinCC_Projects/SRTM_Monitor_v3/panels/
Scripts	/WinCC_Projects/SRTM_Monitor_v3/scripts/
OPC UA Browser Scripts	/home/naherlin/
Node List (CSV)	/home/naherlin/opcua_nodes.csv
Node List (TXT)	/home/naherlin/opcua_nodes.txt

5 Quick Reference Commands

Task	Command
Start Project Admin	startPA
Browse OPC UA nodes	/usr/bin/python3 /browse_opcua.py
Check driver status	ps aux grep -i wincc grep -v grep
Kill project processes	pkill -u \$USER -f 'ProjectName'